BÀI TẬP FULLSTACK – QUẢN LÝ NGƯỜI DÙNG VÀ PHÂN QUYỀN RBAC

MỤC LỤC

I. TỔNG QUAN BÀI TẬP 4
1. Mục tiêu 4
2. Kiến thức áp dụng 4
3. Yêu cầu THỰC HIỆN 4
4. Phạm vi bài tập 4
II. YÊU CẦU CHỨC NĂNG CHI TIẾT 6
1. Mô hình RBAC và cấu trúc dữ liệu 6
1.1. Sơ đồ quan hệ 6
1.2. Permission (danh mục cố định, tạo bằng seed, API chỉ cho phép đọc) 6
1.3. Role 6
1.4. User 7
1.5. Vai trò mặc định (`isSystem = true`) và ma trận quyền 7
1.6. Quy tắc nghiệp vụ bắt buộc 8
2. Quy ước API chung 8
2.1. Thông tin chung 8
2.2. Response thành công 8
2.3. Response lỗi (thống nhất cho mọi lỗi 4xx/5xx) 9
2.4. Mã trạng thái sử dụng 9
3. Backend API Endpoints 10
3.0. Bảng tổng hợp 10
3.1. POST /auth/tokens – Đăng nhập 10
3.2. GET /me – Hồ sơ của tôi 11
3.3. GET /users – Danh sách người dùng 11
3.4. POST /users – Tạo người dùng 12
3.5. GET /users/:userId – Chi tiết 13
3.6. PATCH /users/:userId – Cập nhật một phần 13
3.7. DELETE /users/:userId – Xóa mềm 13

3.8. Vai trò của người dùng 13
3.9. Vai trò 14
3.10. Quyền của vai trò 14
3.11. GET /permissions – Danh mục quyền 14
4. Frontend Requirements 15
4.1. Màn hình Đăng nhập 15
4.2. Bố cục sau khi đăng nhập 15
4.3. Tab Người dùng 16
4.4. Modal Thêm / Sửa người dùng 16
4.5. Modal Gán vai trò 17
4.6. Tab Vai trò và Modal Vai trò 17
4.7. Tab Hồ sơ của tôi 17
4.8. Xử lý lỗi và thông báo 17
4.9. Responsive Design 18
III. HƯỚNG DẪN THỰC HIỆN 19
Bước 1: Backend nền tảng và xác thực (90 phút) 19
1.1. Khởi tạo project 19
1.2. Cấu trúc thư mục Backend 19
1.3. File .env 19
1.4. Models 19
1.5. Tiện ích dùng chung – utils/http.js 21
1.6. Tạo file server.js 22
1.7. Seed dữ liệu – seed.js 24
1.8. Middleware xác thực và phân quyền – middleware/auth.js 25
1.9. Đăng nhập và /me – routes/auth.js 25
1.10. Test Bước 1 với Postman 26
1.11. Dữ liệu tối thiểu 27
Bước 2: Backend RBAC (90 phút) 27
2.1. Khung routes/users.js 27
2.2. Tiện ích phân quyền – utils/rbac.js 28
2.3. Implement GET /users 28
2.4. Implement POST /users 29

2.5. Implement PATCH /users/:userId 29
2.6. Implement DELETE /users/:userId (xóa mềm) 30
2.7. Gán / gỡ vai trò 30
2.8. routes/roles.js và routes/permissions.js 30
2.9. Bổ sung thêm 31
Bước 3: Frontend (120 phút) 31
3.1. Tạo file index.html 31
3.2. Component Structure 32
3.3. Axios instance và interceptor 32
3.4. Implement Component App 33
3.5. Implement Component LoginPage 34
3.6. Implement Component UsersPage 35
3.7. Implement Component UserTable 37
3.8. Implement Component UserModal 37
3.9. Implement Component RoleAssignModal 38
3.10. Implement RolesPage, RoleModal, MyProfile 39
3.11. App Render và CSS 40
3.12. Yêu cầu bổ sung 40
IV. KIỂM THỬ, SẢN PHẨM NỘP VÀ ĐÁNH GIÁ 41
1. Test cases bắt buộc (Postman) 41
2. Kiểm thử tích hợp và bảo mật Frontend 42
3. Sản phẩm nộp 42
V. PHỤ LỤC 43
1. Danh sách mã lỗi 43
2. Lỗi thường gặp khi làm bài 43

I. TỔNG QUAN BÀI TẬP

1. Mục tiêu

Xây dựng ứng dụng web Fullstack có **xác thực** (đăng nhập bằng JWT) và **phân quyền theo vai trò (RBAC – Role-Based Access Control)**:

- Mỗi người dùng có một hoặc nhiều **vai trò** (role); mỗi vai trò gồm nhiều **quyền** (permission).
- Backend kiểm tra quyền ở **mọi** endpoint; Frontend hiển thị chức năng theo quyền của người đang đăng nhập.
- Thiết kế API tuân thủ REST: URI danh từ, đúng HTTP method, đúng mã trạng thái, định dạng JSON và lỗi thống nhất.

2. Kiến thức áp dụng

- **Backend**: Node.js, Express.js, MongoDB, Mongoose, bcryptjs, jsonwebtoken
- **Frontend**: React (CDN), Axios, HTML, CSS
- **Kỹ năng**: REST API, xác thực Bearer token, RBAC, mã trạng thái HTTP (200/201/204/400/401/403/404/409), CORS, Axios interceptor, `async/await`, `Promise.all` / `Promise.allSettled`, phân trang, tìm kiếm

3. Yêu cầu THỰC HIỆN

- **ĐƯỢC PHÉP**: Hỏi AI về lỗi, khái niệm, giải thích syntax
- **KHÔNG ĐƯỢC**: Yêu cầu AI sinh toàn bộ code, copy project từ internet
- Giảng viên sẽ hỏi lại về code để kiểm tra hiểu biết (xem mục IV.4)
- Làm theo cấu trúc dữ liệu, API… theo tài liệu. Giảng viên có thể dùng CSDL khác, Frontend khác hoặc công cụ kiểm thử tự động để chấm Backend, và dùng Backend mẫu để chấm Frontend. **Sai tên trường, sai URI, sai mã trạng thái đều làm hỏng tích hợp.**
- Mã nguồn hướng dẫn trong tài liệu có tính tham khảo, một số phần được giản lược hoặc để `TODO`. Sinh viên lưu ý các yêu cầu bổ sung ở cuối mỗi phần.
- Bắt buộc dùng `async/await` (không dùng chuỗi `.then()`) ở cả Backend và Frontend.

4. Phạm vi bài tập

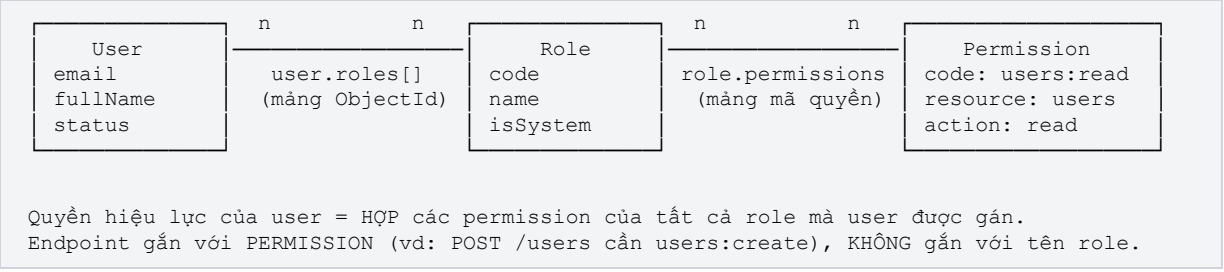
Phần	Bắt buộc	Mở rộng (cộng điểm)
Xác thực	Đăng nhập `POST /auth/tokens`, `GET /me`	Refresh token, đăng xuất phía server, giới hạn số lần đăng nhập sai (`429`)

Phần	Bắt buộc	Mở rộng (cộng điểm)
Người dùng	Danh sách (tìm kiếm, lọc, phân trang), xem, tạo, sửa một phần, khóa/mở khóa, xóa mềm	`PATCH /me`, `PUT /me/password`
Gán vai trò	Xem, gán, gỡ vai trò của người dùng	`GET /users/:userId/permissions` (quyền hiệu lực, nguồn gốc)
Vai trò	Danh sách, xem, tạo, sửa, xóa, thay bộ quyền, xem quyền của vai trò	`GET /roles/:roleId/users`
Danh mục quyền	`GET /permissions`	`ETag` / `304 Not Modified`
Frontend	Đăng nhập, trang Người dùng, trang Vai trò, Hồ sơ của tôi, hiển thị theo quyền	Nhật ký thao tác, triển khai online

II. YÊU CẦU CHỨC NĂNG CHI TIẾT

1. Mô hình RBAC và cấu trúc dữ liệu

1.1. Sơ đồ quan hệ



1.2. Permission (danh mục cố định, tạo bằng seed, API chỉ cho phép đọc)

Mã quyền	Mô tả
`users:read`	Xem danh sách và chi tiết người dùng
`users:create`	Tạo người dùng
`users:update`	Sửa thông tin, khóa/mở khóa người dùng
`users:delete`	Xóa người dùng
`roles:read`	Xem vai trò và quyền của vai trò
`roles:create`	Tạo vai trò
`roles:update`	Sửa vai trò, thay bộ quyền của vai trò
`roles:delete`	Xóa vai trò
`roles:assign`	Gán / gỡ vai trò cho người dùng
`permissions:read`	Xem danh mục quyền

1.3. Role

Trường	Kiểu	Backend Validation	Frontend Validation
`code`	String	required, unique, regex `^[a-z][a-z0-9-]{2,49}$`, **không đổi sau khi tạo**	required, cùng regex
`name`	String	required, trim, 2–100 ký tự	required, trim(), length ≥ 2
`description`	String	không bắt buộc	không bắt buộc
`isSystem`	Boolean	mặc định `false`, không đổi	chỉ hiển thị
`permissions`	[String]	mỗi phần tử phải tồn tại trong danh mục Permission	checkbox theo nhóm resource

1.4. User

Trường	Kiểu	Backend Validation	Frontend Validation
`email`	String	required, unique, trim, lowercase, match `/^\S+@\S+\.\S+$/`	required, đúng định dạng
`fullName`	String	required, trim, 2–100 ký tự	required, trim(), length ≥ 2
`phone`	String	không bắt buộc; nếu có: E.164 `^\+[1-9]\d{7,14}$`	không bắt buộc; dạng `+84912345678`
`passwordHash`	String	bcrypt; `select: false`; **không bao giờ trả ra API**	—
`status`	String	enum `active` | `locked` | `deleted`, mặc định `active`	hiển thị badge
`roles`	[ObjectId]	ref `Role`	hiển thị badge tên vai trò
`lastLoginAt`	Date	cập nhật khi đăng nhập	—
`createdAt`, `updatedAt`	Date	`timestamps: true`	—

Mật khẩu (chỉ nhận khi tạo người dùng): tối thiểu 8 ký tự, có chữ hoa, chữ số và ký tự đặc biệt — regex gợi ý `^(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$`.

1.5. Vai trò mặc định (`isSystem = true`) và ma trận quyền

Permission	`super-admin`	`user-manager`	`auditor`	`member`
`users:read`	✓	✓	✓	
`users:create`	✓	✓		
`users:update`	✓	✓		
`users:delete`	✓			
`roles:read`	✓	✓	✓	
`roles:create`, `roles:update`, `roles:delete`	✓			
`roles:assign`	✓	✓		
`permissions:read`	✓	✓	✓	

Người dùng mới không được chỉ định vai trò sẽ mặc định nhận vai trò `member`.

1.6. Quy tắc nghiệp vụ bắt buộc

Mã	Quy tắc	Vi phạm → mã trạng thái / mã lỗi
BR-01	Email duy nhất (kể cả tài khoản đã xóa mềm); mã vai trò duy nhất	`409 EMAIL_ALREADY_EXISTS` / `409 ROLE_CODE_EXISTS`
BR-02	**Chống leo thang đặc quyền:** chỉ được gán (hoặc gỡ) cho người khác vai trò mà mọi quyền của nó mình đều có; chỉ được cấp cho vai trò những quyền mình có	`403 PRIVILEGE_ESCALATION`
BR-03	Không tự khóa, tự xóa tài khoản của mình; không tự gỡ vai trò của mình	`403 SELF_ACTION_FORBIDDEN`
BR-04	Luôn còn ít nhất 1 người dùng `active` mang vai trò `super-admin`	`409 LAST_SUPER_ADMIN`
BR-05	Vai trò hệ thống không được xóa, không được đổi bộ quyền	`409 SYSTEM_ROLE_PROTECTED`
BR-06	Không xóa vai trò đang được gán cho người dùng (chưa bị xóa)	`409 ROLE_IN_USE`
BR-07	Quyền được tra lại từ CSDL ở **mỗi** request: gỡ vai trò có hiệu lực ngay, không chờ token hết hạn	—
BR-08	Tài khoản `locked` không đăng nhập được; tài khoản `deleted` coi như không tồn tại	`403 ACCOUNT_LOCKED` / `401` / `404`
BR-09	Server **không tin** bất kỳ thông tin phân quyền nào do client gửi lên (role trong body, header tự đặt…)	—

2. Quy ước API chung

2.1. Thông tin chung

Hạng mục	Quy định
Base URL	`http://localhost:3001/api/v1`
Định dạng	JSON, `Content-Type: application/json`
Xác thực	Mọi endpoint (trừ đăng nhập) yêu cầu header `Authorization: Bearer <accessToken>`
ID	Chuỗi ObjectId, trả về ở trường `id` (không trả `_id`, `__v`)
Thời gian	ISO 8601, ví dụ `"2026-10-21T08:07:00.000Z"`
Tên trường	camelCase

2.2. Response thành công

- **Một tài nguyên** → trả về chính đối tượng đó (không bọc thêm `message`).
- **Danh sách** → luôn có dạng:

```
{
  "data": [ { "id": "6717a1c2e4b0a1b2c3d4e5f6", "email":
"an.nguyen@sis.hust.edu.vn" } ],
  "meta": { "page": 1, "limit": 10, "totalItems": 23, "totalPages": 3 }
}
```

- Thao tác không cần trả dữ liệu (xóa, gán/gỡ vai trò) → `204 No Content`, **không có body**.

2.3. Response lỗi (thống nhất cho mọi lỗi 4xx/5xx)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Dữ liệu đầu vào không hợp lệ.",
    "details": [
      { "field": "email", "issue": "INVALID_VALUE", "message": "Email không đúng
định dạng" }
    ]
  }
}
```

- `code`: mã lỗi ổn định để Frontend rẽ nhánh xử lý (danh sách ở Phụ lục).
- `details`: chỉ có khi lỗi gắn với từng trường (validation).
- **Không** trả stack trace, câu truy vấn, đường dẫn file trong response.

2.4. Mã trạng thái sử dụng

Mã	Khi nào
`200 OK`	GET thành công; PATCH/PUT trả về tài nguyên sau cập nhật; đăng nhập thành công
`201 Created`	POST tạo mới thành công, kèm header `Location` trỏ tới tài nguyên mới
`204 No Content`	DELETE; gán / gỡ vai trò
`400 Bad Request`	JSON sai cú pháp, thiếu trường, sai định dạng, gửi trường không được phép, tham số query sai
`401 Unauthorized`	Chưa xác thực: thiếu token, token sai/hết hạn, sai email hoặc mật khẩu
`403 Forbidden`	Đã xác thực nhưng **không đủ quyền**, vượt quyền, tự thao tác trên chính mình, tài khoản bị khóa
`404 Not Found`	Tài nguyên không tồn tại (kể cả ID sai định dạng, người dùng đã xóa mềm); endpoint không tồn tại
`409 Conflict`	Trùng dữ liệu hoặc trạng thái không cho phép (vai trò đang dùng, vai trò hệ thống, super-admin cuối cùng)
`500 Internal Server Error`	Lỗi không lường trước phía server

> Phân biệt **403** và **409**: `403` = *người gọi* không được phép; `409` = *trạng thái dữ liệu* không cho phép (ai gọi cũng bị).

3. Backend API Endpoints

3.0. Bảng tổng hợp

#	Method	URI	Mô tả	Permission	Thành công
A1	`POST`	`/auth/tokens`	Đăng nhập	công khai	`200`
M1	`GET`	`/me`	Hồ sơ + vai trò + quyền hiệu lực của tôi	đã đăng nhập	`200`
U1	`GET`	`/users`	Danh sách (tìm kiếm, lọc, phân trang)	`users:read`	`200`
U2	`POST`	`/users`	Tạo người dùng	`users:create`	`201`
U3	`GET`	`/users/:userId`	Chi tiết	`users:read`	`200`
U4	`PATCH`	`/users/:userId`	Cập nhật một phần, khóa/mở khóa	`users:update`	`200`
U5	`DELETE`	`/users/:userId`	Xóa mềm	`users:delete`	`204`
UR1	`GET`	`/users/:userId/roles`	Vai trò của người dùng	`users:read`	`200`
UR2	`PUT`	`/users/:userId/roles/:roleId`	Gán vai trò (idempotent)	`roles:assign`	`204`
UR3	`DELETE`	`/users/:userId/roles/:roleId`	Gỡ vai trò (idempotent)	`roles:assign`	`204`
R1	`GET`	`/roles`	Danh sách vai trò	`roles:read`	`200`
R2	`POST`	`/roles`	Tạo vai trò	`roles:create`	`201`
R3	`GET`	`/roles/:roleId`	Chi tiết vai trò	`roles:read`	`200`
R4	`PATCH`	`/roles/:roleId`	Sửa tên, mô tả	`roles:update`	`200`
R5	`DELETE`	`/roles/:roleId`	Xóa vai trò	`roles:delete`	`204`
RP1	`GET`	`/roles/:roleId/permissions`	Quyền của vai trò	`roles:read`	`200`
RP2	`PUT`	`/roles/:roleId/permissions`	Thay **toàn bộ** bộ quyền	`roles:update`	`200`
P1	`GET`	`/permissions`	Danh mục quyền	`permissions:read`	`200`

Mọi endpoint cần đăng nhập đều có thể trả `401` (chưa xác thực) và `403 FORBIDDEN` (thiếu permission). Phần chi tiết chỉ liệt kê thêm lỗi đặc thù.

3.1. POST /auth/tokens – Đăng nhập

```
POST /api/v1/auth/tokens
Content-Type: application/json


{ "grantType": "password", "email": "20225000@sis.hust.edu.vn", "password":
"Admin@2026" }
```

Response Success (200):

```
{ "tokenType": "Bearer", "accessToken": "eyJhbGciOiJIUzI1NiIs...", "expiresIn":
3600 }
```

Lỗi	Khi nào
`400 VALIDATION_ERROR`	Thiếu `email`/`password`, `grantType` khác `"password"`
`401 INVALID_CREDENTIALS`	Sai email **hoặc** sai mật khẩu — **cùng một thông điệp** "Email hoặc mật khẩu không đúng."
`403 ACCOUNT_LOCKED`	Đúng thông tin nhưng tài khoản bị khóa

Yêu cầu: mật khẩu chỉ gửi trong body (không đặt trên URL); payload JWT chỉ chứa `sub` (id người dùng) — **không** nhúng danh sách quyền vào token (BR-07).

3.2. GET /me – Hồ sơ của tôi

Response Success (200): biểu diễn User (mục 3.3) bổ sung `permissions` (mảng mã quyền hiệu lực, đã sắp xếp):

```
{
  "id": "6717a1c2e4b0a1b2c3d4e5f6",
  "email": "thanhvien1@sis.hust.edu.vn",
  "fullName": "Trần Thị Thành Viên",
  "phone": null,
  "status": "active",
  "roles": [{ "id": "6717a1c2e4b0a1b2c3d4e001", "code": "user-manager", "name":
"Quản lý người dùng" }],
  "lastLoginAt": "2026-10-21T08:07:00.000Z",
  "createdAt": "2026-10-01T02:00:00.000Z",
  "updatedAt": "2026-10-21T08:07:00.000Z",
  "permissions": ["permissions:read", "roles:assign", "roles:read",
"users:create", "users:read", "users:update"]
}
```

Frontend dùng `permissions` để quyết định ẩn/hiện chức năng.

3.3. GET /users – Danh sách người dùng

Format: `GET /api/v1/users?q=nguyen&status=active&roleId=<id>&sort=-createdAt&page=1&limit=10`

Query	Ý nghĩa	Ràng buộc
`q`	Tìm trong `fullName`, `email` (không phân biệt hoa thường)	tối thiểu 2 ký tự
`status`	Lọc trạng thái	`active` | `locked`
`roleId`	Người dùng có vai trò này	ObjectId hợp lệ
`sort`	Sắp xếp, `-` = giảm dần	chỉ `fullName`, `email`, `createdAt`, `lastLoginAt`; mặc định `-createdAt`
`page`	Trang	số nguyên ≥ 1, mặc định 1
`limit`	Số bản ghi mỗi trang	1–100, mặc định 10

Response Success (200):

```
{
  "data": [
    {
      "id": "6717a1c2e4b0a1b2c3d4e5f6",
      "email": "an.nguyen@sis.hust.edu.vn",
      "fullName": "Nguyễn Văn An",
      "phone": "+84912345678",
      "status": "active",
      "roles": [{ "id": "6717a1c2e4b0a1b2c3d4e004", "code": "member", "name":
"Thành viên" }],
      "lastLoginAt": null,
      "createdAt": "2026-10-20T03:00:00.000Z",
      "updatedAt": "2026-10-20T03:00:00.000Z"
    }
  ],
  "meta": { "page": 1, "limit": 10, "totalItems": 23, "totalPages": 3 }
}
```

Lỗi: `400 INVALID_QUERY_PARAMETER` (ví dụ `limit=500`, `sort=password`, `q=a`), kèm `details` cho từng tham số sai.

Yêu cầu Backend: không trả người dùng `deleted`; dùng `Promise.all` cho `find` và `countDocuments`; thoát ký tự đặc biệt của `q` trước khi đưa vào `$regex`.

3.4. POST /users – Tạo người dùng

```
POST /api/v1/users
Authorization: Bearer <token>
Content-Type: application/json

{
  "email": "le.cuong@sis.hust.edu.vn",
  "fullName": "Lê Văn Cường",
  "phone": "+84901112233",
  "password": "Init@2026",
  "roleIds": ["6717a1c2e4b0a1b2c3d4e004"]
}
```

Response Success (201) — header `Location: /api/v1/users/<id>`, body là User vừa tạo (không có mật khẩu).

Lỗi	Khi nào
`400 VALIDATION_ERROR`	Thiếu/sai trường; mật khẩu yếu (`issue: "WEAK_PASSWORD"`); gửi trường lạ như `status`, `isAdmin` (`issue: "FIELD_NOT_ALLOWED"`)
`400 ROLE_NOT_FOUND`	Có `roleId` không tồn tại
`403 FORBIDDEN`	Có `roleIds` nhưng người gọi không có `roles:assign`
`403 PRIVILEGE_ESCALATION`	Vai trò được gán có quyền mà người gọi không có (BR-02)
`409 EMAIL_ALREADY_EXISTS`	Email đã tồn tại

3.5. GET /users/:userId – Chi tiết

`200` — đối tượng User. `404 USER_NOT_FOUND` nếu không tồn tại, đã xóa mềm, hoặc `userId` không phải ObjectId hợp lệ.

3.6. PATCH /users/:userId – Cập nhật một phần

Chỉ nhận các trường: `fullName`, `phone`, `email`, `status` (`active` \| `locked`). **Chỉ cập nhật trường được gửi lên**; trường không gửi giữ nguyên.

```
{ "status": "locked" }
```

Response Success (200) — User sau cập nhật.

Lỗi	Khi nào
`400 VALIDATION_ERROR`	Giá trị sai; gửi `roles`, `password`, `id`… (`FIELD_NOT_ALLOWED`)
`403 SELF_ACTION_FORBIDDEN`	Tự khóa chính mình
`404 USER_NOT_FOUND`	
`409 EMAIL_ALREADY_EXISTS`	Đổi sang email đã tồn tại
`409 LAST_SUPER_ADMIN`	Khóa super-admin active cuối cùng

> Vì sao dùng **PATCH** chứ không phải **PUT**? PUT có ngữ nghĩa *thay thế toàn bộ* tài nguyên; ở đây ta chỉ sửa một vài trường.

3.7. DELETE /users/:userId – Xóa mềm

Response Success (204) — không có body. Đặt `status = "deleted"`; sau đó `GET /users/:userId` → `404`, người dùng không đăng nhập được.

Lỗi: `403 SELF_ACTION_FORBIDDEN`, `404 USER_NOT_FOUND`, `409 LAST_SUPER_ADMIN`.

3.8. Vai trò của người dùng

GET /users/:userId/roles → `200`

```
{ "data": [{ "id": "6717...e001", "code": "user-manager", "name": "Quản lý người dùng" }], "meta": { "totalItems": 1 } }
```

PUT /users/:userId/roles/:roleId — gán vai trò, **không có body** → `204`. Gọi lại nhiều lần vẫn `204` và **không** tạo bản ghi trùng.

Lỗi: `403 PRIVILEGE_ESCALATION`, `404 USER_NOT_FOUND`, `404 ROLE_NOT_FOUND`.

DELETE /users/:userId/roles/:roleId — gỡ vai trò → `204` (kể cả khi người dùng vốn không có vai trò đó).

Lỗi: `403 SELF_ACTION_FORBIDDEN`, `403 PRIVILEGE_ESCALATION`, `404`, `409 LAST_SUPER_ADMIN`.

3.9. Vai trò

GET /roles — query `q`, `page`, `limit`, `sort` (`name` | `code` | `createdAt`, mặc định `name`) → `200` dạng danh sách, mỗi phần tử:

```
{
  "id": "6717a1c2e4b0a1b2c3d4e005",
  "code": "tro-giang",
  "name": "Trợ giảng",
  "description": "Hỗ trợ xem danh sách người dùng",
  "isSystem": false,
  "permissions": ["users:read", "roles:read"],
  "userCount": 2,
  "createdAt": "2026-10-21T08:00:00.000Z",
  "updatedAt": "2026-10-21T08:00:00.000Z"
}
```

`userCount` = số người dùng chưa bị xóa đang mang vai trò.

POST /roles — body `{ "code", "name", "description", "permissions" }` → `201` + `Location`.

Lỗi: `400 VALIDATION_ERROR`, `400 PERMISSION_NOT_FOUND`, `403 PRIVILEGE_ESCALATION`, `409 ROLE_CODE_EXISTS`.

GET /roles/:roleId → `200`; `404 ROLE_NOT_FOUND`.

PATCH /roles/:roleId — chỉ nhận `name`, `description` (gửi `code`, `isSystem`, `permissions` → `400 FIELD_NOT_ALLOWED`) → `200`.

DELETE /roles/:roleId → `204`. Lỗi: `404`, `409 SYSTEM_ROLE_PROTECTED`, `409 ROLE_IN_USE`.

3.10. Quyền của vai trò

GET /roles/:roleId/permissions → `200`:

```
{ "data": [{ "code": "users:read", "resource": "users", "action": "read",
"description": "Xem danh sách và chi tiết người dùng" }], "meta": { "totalItems":
1 } }
```

PUT /roles/:roleId/permissions — thay **toàn bộ** bộ quyền:

```
{ "permissions": ["users:read", "users:update", "roles:read"] }
```

→ `200`, body là Role sau cập nhật. Lỗi: `400 PERMISSION_NOT_FOUND`, `403 PRIVILEGE_ESCALATION` (thêm/bớt quyền mình không có), `404`, `409 SYSTEM_ROLE_PROTECTED`.

3.11. GET /permissions – Danh mục quyền

Query `resource` (ví dụ `?resource=users`) → `200` dạng `{ data, meta }`, sắp xếp theo `resource`, `code`.

4. Frontend Requirements

4.1. Màn hình Đăng nhập

```
┌──────────────────────────────┐
│ Đăng nhập                    │
│ Email:    [_____________]    │
│ Mật khẩu: [_____________]    │
│ (thông báo lỗi màu đỏ)       │
│                [Đăng nhập]   │
└──────────────────────────────┘
```

- Gọi `POST /auth/tokens`; thành công → lưu token, gọi `GET /me`, vào màn hình chính.
- Hiển thị thông báo của server cho `401`, `403`; thông báo riêng khi không kết nối được (server tắt, CORS).

4.2. Bố cục sau khi đăng nhập

```
┌──────────────────────────────────────────────────────────┐
│ Quản lý Người dùng & Phân quyền              [Đăng xuất] │
│ Trần Thị Thành Viên (thanhvien1@...) · [Quản lý người dùng] │
├──────────────────────────────────────────────────────────┤
│ [Người dùng] [Vai trò] [Hồ sơ của tôi]                   │
├──────────────────────────────────────────────────────────┤
│                  (nội dung tab đang chọn)                │
└──────────────────────────────────────────────────────────┘
```

Hiển thị theo quyền (dựa vào `me.permissions`):

Thành phần giao diện	Hiện khi có quyền
Tab "Người dùng"	`users:read`
Tab "Vai trò"	`roles:read`
Tab "Hồ sơ của tôi"	luôn hiện
Nút "+ Thêm người dùng"	`users:create`
Nút "Sửa", "Khóa / Mở khóa"	`users:update`
Nút "Vai trò" (gán vai trò)	`roles:assign` và `roles:read`
Nút "Xóa" người dùng	`users:delete`
Bộ lọc theo vai trò	`roles:read`
Nút "+ Thêm vai trò"	`roles:create`
Nút "Sửa" vai trò	`roles:update`
Nút "Xóa" vai trò	`roles:delete` và vai trò không phải hệ thống

> ⚠️ Ẩn nút chỉ để **cải thiện trải nghiệm**. Bảo mật thực sự nằm ở Backend: người dùng vẫn có thể gọi API trực tiếp bằng Postman/DevTools.

4.3. Tab Người dùng

```
┌──────────────────────────────────────────────────────────────────────┐
│ [Tìm theo họ tên, email...] [Trạng thái v] [Vai trò v] [+ Thêm]      │
├──────┬──────────┬──────────┬──────────────┬──────────┬──────────────┤
│ STT  │ Họ tên   │ Email    │ Vai trò      │ Trạng thái │ Thao tác   │
│ 1    │ Nguyễn   │ nva@...  │ [Thành viên] │ [Hoạt động]│ Sửa  Vai trò │
│      │          │          │              │            │ Khóa Xóa   │
├──────┴──────────┴──────────┴──────────────┴──────────┴──────────────┤
│ Hiển thị: [5 v] dòng/trang · Tổng 23      [Prev] Trang 1/5 [Next]   │
└──────────────────────────────────────────────────────────────────────┘
```

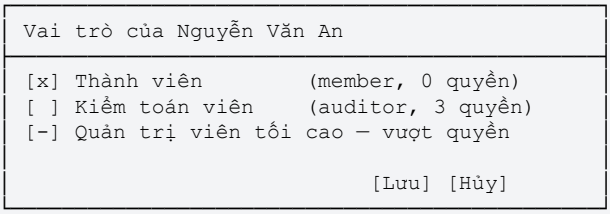
- STT tăng liên tục theo trang (trang 2 với 5 dòng/trang bắt đầu từ 6).
- Tìm kiếm phía Backend, debounce 400ms; chỉ gửi `q` khi có ≥ 2 ký tự.
- Đổi từ khóa, bộ lọc hoặc số dòng/trang → quay về trang 1.
- Dropdown số dòng/trang: 5, 10, 20. Prev disabled ở trang 1, Next disabled ở trang cuối.
- Dòng của chính người đang đăng nhập: hiển thị "(bạn)", nút Khóa và Xóa bị **disabled** (BR-03).
- Xóa: hộp thoại xác nhận trước khi gọi API.
- **Sau mỗi thao tác thay đổi dữ liệu phải gọi lại API danh sách** để cập nhật giao diện.

4.4. Modal Thêm / Sửa người dùng

```
┌──────────────────────────────────────┐
│ Thêm người dùng              [X]     │
├──────────────────────────────────────┤
│ Email *:        [_____________]      │
│ Họ tên *:       [_____________]      │
│ Số điện thoại:  [_____________]      │
│ Mật khẩu *:     [_____________]      │  <- chỉ khi thêm mới
│ Vai trò:  [ ] Thành viên             │  <- chỉ khi thêm mới + roles:assign
│           [ ] Kiểm toán viên         │
│           [-] Quản trị viên tối cao  │  <- disabled: vượt quyền của bạn
│                                      │
│                    [Lưu] [Hủy]       │
└──────────────────────────────────────┘
```

- Validation Frontend theo bảng 1.4; lỗi hiển thị màu đỏ dưới từng ô.
- Chuẩn hóa trước khi gửi: `trim()` mọi chuỗi, email chữ thường, số điện thoại rỗng → `null`.
- Sửa: chỉ gửi **các trường đã thay đổi** bằng `PATCH`.
- Lỗi `400` từ server: hiển thị `details[].message` dưới đúng ô theo `details[].field`; lỗi `409 EMAIL_ALREADY_EXISTS` hiển thị dưới ô Email.
- Vai trò có quyền vượt quá quyền của người đang đăng nhập: checkbox **disabled** kèm ghi chú.

4.5. Modal Gán vai trò



- Khi Lưu: so sánh với vai trò ban đầu → danh sách cần **gán** (`PUT`) và cần **gỡ** (`DELETE`).
- Gửi các request **song song** bằng `Promise.allSettled`; báo cụ thể vai trò nào thất bại và lý do, các thao tác thành công vẫn được giữ.
- Khi mở modal cho chính mình: không cho bỏ chọn vai trò đang có (BR-03).

4.6. Tab Vai trò và Modal Vai trò

Mã	Tên (+ mô tả)	Quyền (badge)	Người dùng	Loại	Thao tác
`user-manager`	Quản lý người dùng	`users:read` `users:create` …	3	Hệ thống	Sửa
`tro-giang`	Trợ giảng	`users:read` `roles:read`	2	Tùy chỉnh	Sửa Xóa

Modal Thêm/Sửa vai trò:

- Ô "Mã vai trò" chỉ có khi thêm mới (mã không đổi được).
- Danh sách quyền lấy từ `GET /permissions`, **nhóm theo `resource`**, mỗi quyền là một checkbox.
- Vai trò hệ thống: toàn bộ checkbox quyền bị disabled; quyền mà người đang đăng nhập không có: disabled.
- Lưu khi thêm: `POST /roles`. Lưu khi sửa: `PATCH /roles/:roleId` nếu tên/mô tả đổi, rồi `PUT /roles/:roleId/permissions` nếu bộ quyền đổi.
- Xóa vai trò: xác nhận; lỗi `409` hiển thị thông điệp của server (vai trò đang dùng / vai trò hệ thống).

4.7. Tab Hồ sơ của tôi

Hiển thị họ tên, email, số điện thoại, vai trò, danh sách quyền hiệu lực (từ `GET /me`). Thành viên không có quyền quản trị chỉ thấy tab này.

4.8. Xử lý lỗi và thông báo

Tình huống	Hành vi giao diện
Thành công	Thông báo xanh ở góc màn hình (tự ẩn sau ~4 giây)

Tình huống	Hành vi giao diện
`400`	Hiển thị lỗi dưới từng ô (nếu có `details`), hoặc thông báo đỏ
`401` (không phải lúc đăng nhập)	Xóa token, quay về màn hình đăng nhập, báo "Phiên đăng nhập đã hết hạn" — xử lý **tập trung** trong Axios interceptor
`403`	Thông báo đỏ với thông điệp của server; tải lại `/me` để đồng bộ quyền
`404`	Thông báo, tải lại danh sách
`409`	Thông báo đỏ với thông điệp của server
Lỗi mạng / CORS / timeout	"Không kết nối được máy chủ…"

4.9. Responsive Design

Breakpoint: 768px. Desktop: bảng đầy đủ, modal rộng 460px ở giữa. Mobile: bảng cuộn ngang (`overflow-x: auto`), modal rộng 95%, chữ nhỏ hơn, nút tối thiểu 44×44px.

III. HƯỚNG DẪN THỰC HIỆN

Bước 1: Backend nền tảng và xác thực (90 phút)

1.1. Khởi tạo project

```
mkdir rbac-app
cd rbac-app
mkdir backend frontend
cd backend
npm init -y
npm install express mongoose cors bcryptjs jsonwebtoken dotenv
mkdir models middleware routes utils
```

> `npm install express` hiện cài Express 5: lỗi trong hàm `async` của route được tự chuyển tới error handler chung, nên không cần bọc `try/catch` ở mọi route.

1.2. Cấu trúc thư mục Backend

```
backend/
├── .env                    Cấu hình (KHÔNG commit lên Git)
├── server.js               Khởi tạo app, CORS, routes, error handler
├── seed.js                 Tạo dữ liệu mẫu
├── models/
│   ├── Permission.js
│   ├── Role.js
│   └── User.js
├── middleware/
│   └── auth.js             authenticate, requirePermission
├── routes/
│   ├── auth.js             /auth/tokens, /me
│   ├── users.js            /users/...
│   ├── roles.js            /roles/...
│   └── permissions.js      /permissions
└── utils/
    ├── http.js             fail(), phân trang, biểu diễn tài nguyên
    └── rbac.js             chống leo thang đặc quyền, super-admin cuối cùng
```

1.3. File .env

```
PORT=3001
# Kết nối MongoDB với username là MSSV, password là MSSV, dbname là it4409_rbac
MONGODB_URI=mongodb+srv://<MSSV>:<MSSV>@cluster.mongodb.net/it4409_rbac
JWT_SECRET=<chuỗi ngẫu nhiên dài, tự đặt>
ACCESS_TOKEN_TTL=3600
CORS_ORIGINS=http://127.0.0.1:5500,http://localhost:5500
```

> Dùng database **khác** Ex3 (`it4409_rbac`) vì collection `users` có cấu trúc khác.

1.4. Models

models/Permission.js

```
const mongoose = require("mongoose");


const permissionSchema = new mongoose.Schema({
  code: { type: String, required: true, unique: true },   // "users:read"
  resource: { type: String, required: true },             // "users"
  action: { type: String, required: true },               // "read"
  description: { type: String, default: "" },
});
```

```
module.exports = mongoose.model("Permission", permissionSchema);
```

models/Role.js

```
const mongoose = require("mongoose");

const roleSchema = new mongoose.Schema(
  {
    code: {
      type: String,
      required: [true, "Mã role không được để trống"],
      unique: true,
      trim: true,
      match: [/^[a-z][a-z0-9-]{2,49}$/, "Mã role: chữ thường, số, gạch nối; 3-50
ký tự; bắt đầu bằng chữ"],
      immutable: true, // không đổi sau khi tạo
    },
    name: {
      type: String,
      required: [true, "Tên role không được để trống"],
      trim: true,
      minlength: [2, "Tên role phải có ít nhất 2 ký tự"],
      maxlength: [100, "Tên role tối đa 100 ký tự"],
    },
    description: { type: String, default: null },
    isSystem: { type: Boolean, default: false, immutable: true },
    permissions: { type: [String], default: [] },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Role", roleSchema);
```

models/User.js (bắt buộc)

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    email: {
      type: String,
      required: [true, "Email không được để trống"],
      unique: true,
      trim: true,
      lowercase: true,
      match: [/^\S+@\S+\.\S+$/, "Email không đúng định dạng"],
    },
    fullName: {
      type: String,
      required: [true, "Họ tên không được để trống"],
      trim: true,
      minlength: [2, "Họ tên phải có ít nhất 2 ký tự"],
      maxlength: [100, "Họ tên tối đa 100 ký tự"],
    },
    phone: {
      type: String,
      default: null,
      match: [/^\+[1-9]\d{7,14}$/, "Số điện thoại theo định dạng E.164, ví dụ
+84912345678"],
    },
    passwordHash: { type: String, required: true, select: false }, // không bao
giờ trả ra ngoài
```

```
    status: { type: String, enum: ["active", "locked", "deleted"], default:
"active" },
    roles: [{ type: mongoose.Schema.Types.ObjectId, ref: "Role" }], // quan hệ n:n
User–Role
    lastLoginAt: { type: Date, default: null },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

Giải thích:

- `select: false`: mọi truy vấn mặc định không lấy `passwordHash`; chỉ lấy khi cần bằng `.select("+passwordHash")` (lúc đăng nhập).
- `roles: [ObjectId]` với `ref: "Role"`: lưu quan hệ nhiều–nhiều ngay trong User; `.populate("roles")` để lấy đầy đủ thông tin vai trò.
- `immutable: true`: Mongoose bỏ qua mọi thay đổi với trường này sau khi tạo.

1.5. Tiện ích dùng chung – utils/http.js

```
const mongoose = require("mongoose");

// Gửi lỗi theo định dạng thống nhất { error: { code, message, details? } }
function fail(res, status, code, message, details) {
  const error = { code, message };
  if (details && details.length) error.details = details;
  return res.status(status).json({ error });
}

const detail = (field, issue, message) => ({ field, issue, message });

// Lỗi cho các trường không nằm trong allow-list (chống mass assignment)
function notAllowedFields(body, allowed) {
  return Object.keys(body || {})
    .filter((k) => !allowed.includes(k))
    .map((k) => detail(k, "FIELD_NOT_ALLOWED", `Trường '${k}' không được phép gửi
lên endpoint này.`));
}

const isObjectId = (id) => mongoose.isValidObjectId(id) && String(new
mongoose.Types.ObjectId(id)) === String(id);

// Đọc và kiểm tra page/limit
function parsePaging(query, details) {
  const page = query.page === undefined ? 1 : Number(query.page);
  const limit = query.limit === undefined ? 10 : Number(query.limit);
  if (!Number.isInteger(page) || page < 1) details.push(detail("page",
"INVALID_VALUE", "page phải là số nguyên ≥ 1."));
  if (!Number.isInteger(limit) || limit < 1 || limit > 100)
details.push(detail("limit", "OUT_OF_RANGE", "limit phải trong khoảng 1–100."));
  return { page, limit };
}

// sort=-createdAt,fullName  ->  { createdAt: -1, fullName: 1 }
function parseSort(raw, allowed, details) {
  const sort = {};
  for (const part of String(raw).split(",").map((s) => s.trim()).filter(Boolean))
{
```

```
    const field = part.replace(/^-/, "");
    if (!allowed.includes(field)) {
      details.push(detail("sort", "FIELD_NOT_SORTABLE", `Không sắp xếp được theo
'${field}'.`));
      continue;
    }
    sort[field] = part.startsWith("-") ? -1 : 1;
  }
  return sort;
}

// Thoát ký tự đặc biệt trước khi đưa từ khóa vào $regex
const escapeRegex = (s) => String(s).replace(/[.*+?^${}()|[\]\\]/g, "\\$&");

const pageMeta = (page, limit, totalItems) => ({ page, limit, totalItems,
totalPages: Math.max(1, Math.ceil(totalItems / limit)) });

// ---- Biểu diễn tài nguyên: CHỈ chọn trường được phép trả ra ----
const roleSummary = (r) => ({ id: String(r._id), code: r.code, name: r.name });

function userView(u) {
  return {
    id: String(u._id),
    email: u.email,
    fullName: u.fullName,
    phone: u.phone,
    status: u.status,
    roles: (u.roles || []).filter((r) => r && r.code).map(roleSummary),
    lastLoginAt: u.lastLoginAt,
    createdAt: u.createdAt,
    updatedAt: u.updatedAt,
  };
}

function roleView(r, userCount) {
  return {
    id: String(r._id), code: r.code, name: r.name, description: r.description,
    isSystem: r.isSystem, permissions: [...r.permissions], userCount,
    createdAt: r.createdAt, updatedAt: r.updatedAt,
  };
}

module.exports = { fail, detail, notAllowedFields, isObjectId, parsePaging,
parseSort, escapeRegex, pageMeta, userView, roleView, roleSummary };
```

> Vì sao viết `userView()` thay vì `res.json(user)`? Để kiểm soát chính xác trường nào được trả ra (không lộ `passwordHash`, `__v`), đổi `_id` → `id`, và giữ cấu trúc response ổn định cho Frontend.

1.6. Tạo file server.js

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const { fail, detail } = require("./utils/http");

const app = express();

// CORS: chỉ cho phép các origin trong danh sách trắng (KHÔNG dùng cors() mở cho
mọi origin)
```

```
const allowed = (process.env.CORS_ORIGINS || "").split(",").map((s) =>
s.trim()).filter(Boolean);
app.use(cors({
  origin: (origin, cb) => cb(null, !origin || allowed.includes(origin)),
  methods: ["GET", "POST", "PUT", "PATCH", "DELETE"],
  allowedHeaders: ["Authorization", "Content-Type"],
  exposedHeaders: ["Location"],
}));
app.use(express.json({ limit: "100kb" }));

// Routes
app.use("/api/v1", require("./routes/auth"));          // /auth/tokens, /me
app.use("/api/v1/users", require("./routes/users"));
app.use("/api/v1/roles", require("./routes/roles"));
app.use("/api/v1/permissions", require("./routes/permissions"));

// 404 cho endpoint không tồn tại
app.use((req, res) => fail(res, 404, "ENDPOINT_NOT_FOUND", `Không có endpoint
${req.method} ${req.path}.`));

// Error handler chung
app.use((err, req, res, next) => {
  if (err.type === "entity.parse.failed") return fail(res, 400, "MALFORMED_JSON",
"Body không phải JSON hợp lệ.");
  if (err.name === "ValidationError") {
    const details = Object.values(err.errors).map((e) =>
      detail(e.path, e.kind === "required" ? "REQUIRED" : "INVALID_VALUE",
e.message));
    return fail(res, 400, "VALIDATION_ERROR", "Dữ liệu đầu vào không hợp lệ.",
details);
  }
  if (err.code === 11000) {
    // TODO: xác định trường bị trùng từ err.keyPattern -> EMAIL_ALREADY_EXISTS /
ROLE_CODE_EXISTS
    return fail(res, 409, "CONFLICT", "Dữ liệu bị trùng.");
  }
  console.error(err); // chi tiết chỉ ghi log phía server
  fail(res, 500, "INTERNAL_ERROR", "Đã có lỗi phía server. Vui lòng thử lại
sau.");
});

const PORT = Number(process.env.PORT || 3001);
mongoose
  .connect(process.env.MONGODB_URI)
  .then(() => {
    console.log("Connected to MongoDB");
    app.listen(PORT, () => console.log(`Server running on
http://localhost:${PORT}`));
  })
  .catch((err) => {
    console.error("MongoDB Error:", err.message);
    process.exit(1);
  });
```

Giải thích:

- `exposedHeaders: ["Location"]`: cho phép JavaScript ở origin khác đọc header `Location` sau `201`.
- Error handler chung chuyển lỗi của Mongoose thành mã HTTP đúng: lỗi validation → `400`, trùng unique index (`11000`) → `409`. Mọi lỗi khác → `500` với thông điệp chung.

1.7. Seed dữ liệu – seed.js

```
require("dotenv").config();
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const Permission = require("./models/Permission");
const Role = require("./models/Role");
const User = require("./models/User");

const PERMISSIONS = [
  ["users:read", "Xem danh sách và chi tiết người dùng"],
  ["users:create", "Tạo người dùng"],
  ["users:update", "Sửa thông tin, khóa/mở khóa người dùng"],
  ["users:delete", "Xóa người dùng"],
  ["roles:read", "Xem role và quyền của role"],
  ["roles:create", "Tạo role"],
  ["roles:update", "Sửa role, thay bộ quyền của role"],
  ["roles:delete", "Xóa role"],
  ["roles:assign", "Gán / gỡ role cho người dùng"],
  ["permissions:read", "Xem danh mục quyền"],
].map(([code, description]) => ({ code, resource: code.split(":")[0], action:
code.split(":")[1], description }));

const ROLES = [
  { code: "super-admin", name: "Quản trị viên tối cao", isSystem: true,
permissions: PERMISSIONS.map((p) => p.code) },
  // TODO: user-manager, auditor, member theo ma trận ở mục II.1.5
];

// TODO: dữ liệu THẬT theo yêu cầu "Dữ liệu tối thiểu" (mục 1.11)
const USERS = [
  { email: "<MSSV>@sis.hust.edu.vn", fullName: "<Họ tên của bạn>", phone:
"+84...", password: "Admin@2026", roles: ["super-admin"] },
  // ...
];

(async () => {
  await mongoose.connect(process.env.MONGODB_URI);
  await Promise.all([Permission.deleteMany({}), Role.deleteMany({}),
User.deleteMany({})]);
  await Promise.all([Permission.init(), Role.init(), User.init()]); // tạo unique
index

  await Permission.insertMany(PERMISSIONS);
  const roles = await Role.insertMany(ROLES);
  const roleId = Object.fromEntries(roles.map((r) => [r.code, r._id]));

  for (const u of USERS) {
    await User.create({
      email: u.email, fullName: u.fullName, phone: u.phone ?? null,
      passwordHash: await bcrypt.hash(u.password, 10),
      roles: u.roles.map((code) => roleId[code]),
    });
  }
  console.log(`Đã tạo ${PERMISSIONS.length} quyền, ${roles.length} role,
${USERS.length} người dùng.`);
  await mongoose.disconnect();
})();
```

Chạy: `node seed.js` (lệnh này **xóa** dữ liệu cũ của 3 collection rồi tạo lại).

1.8. Middleware xác thực và phân quyền – middleware/auth.js

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");
const { fail } = require("../utils/http");

// Xác thực: đọc Bearer token, kiểm tra chữ ký, nạp user và quyền hiệu lực
async function authenticate(req, res, next) {
  const match = (req.get("Authorization") || "").match(/^Bearer\s+(.+)$/i);
  if (!match) return fail(res, 401, "UNAUTHENTICATED", "Thiếu access token. Gửi
header Authorization: Bearer <token>.");

  let payload;
  try {
    payload = jwt.verify(match[1], process.env.JWT_SECRET);
  } catch (err) {
    return err.name === "TokenExpiredError"
      ? fail(res, 401, "TOKEN_EXPIRED", "Access token đã hết hạn. Vui lòng đăng
nhập lại.")
      : fail(res, 401, "INVALID_TOKEN", "Access token không hợp lệ.");
  }

  const user = await User.findById(payload.sub).populate("roles");
  if (!user || user.status !== "active") return fail(res, 401, "INVALID_TOKEN",
"Phiên đăng nhập không còn hiệu lực.");

  // Quyền hiệu lực = HỢP các permission của mọi role, tra lại ở MỖI request (BR-
07)
  req.user = user;
  req.permissions = new Set(user.roles.flatMap((r) => r.permissions));
  next();
}

// Phân quyền: endpoint khai báo PERMISSION cần có, không so sánh tên role
function requirePermission(...codes) {
  return (req, res, next) => {
    const missing = codes.filter((c) => !req.permissions.has(c));
    if (missing.length) return fail(res, 403, "FORBIDDEN", `Bạn không có quyền
thực hiện thao tác này (cần: ${missing.join(", ")}).`);
    next();
  };
}

module.exports = { authenticate, requirePermission };
```

Giải thích:

- Token do server **ký**; client không thể sửa nội dung token mà không làm hỏng chữ ký.
- Token chỉ cho biết **ai** đang gọi. **Quyền** luôn tra từ CSDL, nên khi quản trị viên gỡ vai trò, request tiếp theo bị chặn ngay.
- Không bao giờ viết `if (req.user.role === "admin")` trong từng route: thêm vai trò mới sẽ phải sửa code.

1.9. Đăng nhập và /me – routes/auth.js

```
const express = require("express");
const bcrypt = require("bcryptjs");
```

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");
const { authenticate } = require("../middleware/auth");
const { fail, detail, userView } = require("../utils/http");

const router = express.Router();
const ACCESS_TTL = Number(process.env.ACCESS_TOKEN_TTL || 3600);

router.post("/auth/tokens", async (req, res) => {
  const { grantType, email, password } = req.body || {};
  const details = [];
  if (grantType !== "password") details.push(detail("grantType", "UNSUPPORTED",
"grantType phải là 'password'."));
  if (!email) details.push(detail("email", "REQUIRED", "Thiếu email."));
  if (!password) details.push(detail("password", "REQUIRED", "Thiếu mật khẩu."));
  if (details.length) return fail(res, 400, "VALIDATION_ERROR", "Dữ liệu đầu vào
không hợp lệ.", details);

  const user = await User.findOne({ email: String(email).trim().toLowerCase(),
status: { $ne: "deleted" } })
    .select("+passwordHash");
  const ok = user && (await bcrypt.compare(String(password), user.passwordHash));
  if (!ok) return fail(res, 401, "INVALID_CREDENTIALS", "Email hoặc mật khẩu không
đúng.");
  // TODO: tài khoản locked -> 403 ACCOUNT_LOCKED

  user.lastLoginAt = new Date();
  await user.save();

  const accessToken = jwt.sign({ sub: String(user._id) }, process.env.JWT_SECRET,
{ expiresIn: ACCESS_TTL });
  res.set("Cache-Control", "no-store");
  res.json({ tokenType: "Bearer", accessToken, expiresIn: ACCESS_TTL });
});

router.get("/me", authenticate, (req, res) => {
  res.json({ ...userView(req.user), permissions: [...req.permissions].sort() });
});

module.exports = router;
```

Câu hỏi tự kiểm tra: Vì sao sai email và sai mật khẩu phải trả **cùng** một thông điệp?

1.10. Test Bước 1 với Postman

1. `POST /auth/tokens` đúng thông tin → `200`, có `accessToken`
2. `POST /auth/tokens` sai mật khẩu / sai email → `401`, cùng thông điệp
3. `POST /auth/tokens` thiếu `password` → `400`, có `details`
4. `GET /me` không có header `Authorization` → `401 UNAUTHENTICATED`
5. `GET /me` với token sửa 1 ký tự → `401 INVALID_TOKEN`
6. `GET /me` đúng token → `200`, có `permissions`, **không** có `passwordHash`

Gợi ý Postman: trong tab *Scripts → Post-response* của request đăng nhập, lưu token vào biến: `pm.collectionVariables.set("token", pm.response.json().accessToken);`, các request khác dùng *Authorization → Bearer Token* = `{{token}}`.

1.11. Dữ liệu tối thiểu

- Bản thân ở vị trí đầu tiên, vai trò `super-admin` (chính xác họ tên, email sinh viên)
- Các thành viên trong nhóm BTL: ít nhất 1 `user-manager`, 1 `auditor` (chính xác tên, email)
- Các bạn ngồi xung quanh: vai trò `member` (chính xác tên)
- Ít nhất 1 vai trò tự tạo (không phải hệ thống), được gán cho ít nhất 1 người

Bước 2: Backend RBAC (90 phút)

2.1. Khung routes/users.js

```
const express = require("express");
const bcrypt = require("bcryptjs");
const User = require("../models/User");
const Role = require("../models/Role");
const { authenticate, requirePermission } = require("../middleware/auth");
const { missingPermissions, isLastSuperAdmin } = require("../utils/rbac");
const { fail, detail, notAllowedFields, isObjectId, parsePaging, parseSort,
escapeRegex, pageMeta, userView, roleSummary } = require("../utils/http");

const router = express.Router();
router.use(authenticate); // mọi endpoint dưới đây đều cần đăng nhập

// Middleware nạp user theo :userId (bỏ qua user đã xóa mềm)
async function loadUser(req, res, next) {
  const { userId } = req.params;
  const user = isObjectId(userId)
    ? await User.findOne({ _id: userId, status: { $ne: "deleted" }
}).populate("roles")
    : null;
  if (!user) return fail(res, 404, "USER_NOT_FOUND", "Không tìm thấy người
dùng.");
  req.target = user;
  next();
}
// TODO: loadRole tương tự, trả 404 ROLE_NOT_FOUND

router.get("/", requirePermission("users:read"), async (req, res) => { /* 2.3 */
});
router.post("/", requirePermission("users:create"), async (req, res) => { /* 2.4
*/ });
router.get("/:userId", requirePermission("users:read"), loadUser, (req, res) =>
res.json(userView(req.target)));
router.patch("/:userId", requirePermission("users:update"), loadUser, async (req,
res) => { /* 2.5 */ });
router.delete("/:userId", requirePermission("users:delete"), loadUser, async (req,
res) => { /* 2.6 */ });
router.get("/:userId/roles", requirePermission("users:read"), loadUser, (req, res)
=> { /* TODO */ });
router.put("/:userId/roles/:roleId", requirePermission("roles:assign"), loadUser,
loadRole, async (req, res) => { /* 2.7 */ });
router.delete("/:userId/roles/:roleId", requirePermission("roles:assign"),
loadUser, loadRole, async (req, res) => { /* 2.7 */ });

module.exports = router;
```

Chú ý thứ tự middleware: **xác thực → phân quyền → nạp tài nguyên → xử lý**.

2.2. Tiện ích phân quyền – utils/rbac.js

```
const Role = require("../models/Role");
const User = require("../models/User");

// Những quyền trong danh sách mà người gọi KHÔNG có (dùng chống leo thang đặc
quyền)
const missingPermissions = (codes, myPermissions) => [...new
Set(codes)].filter((c) => !myPermissions.has(c));

// user có phải super-admin active cuối cùng không
async function isLastSuperAdmin(user) {
  const sa = await Role.findOne({ code: "super-admin" });
  if (!sa || !user.roles.some((r) => String(r._id || r) === String(sa._id)))
return false;
  const others = await User.countDocuments({ _id: { $ne: user._id }, status:
"active", roles: sa._id });
  return others === 0;
}

module.exports = { missingPermissions, isLastSuperAdmin };
```

2.3. Implement GET /users

```
router.get("/", requirePermission("users:read"), async (req, res) => {
  const q = req.query;
  const details = [];
  const { page, limit } = parsePaging(q, details);
  const sort = parseSort(q.sort || "-createdAt", ["fullName", "email",
"createdAt", "lastLoginAt"], details);
  if (q.status && !["active", "locked"].includes(q.status))
details.push(detail("status", "INVALID_VALUE", "status phải là active hoặc
locked."));
  // TODO: kiểm tra roleId (ObjectId hợp lệ) và q (tối thiểu 2 ký tự)
  if (details.length) return fail(res, 400, "INVALID_QUERY_PARAMETER", "Tham số
truy vấn không hợp lệ.", details);

  const filter = { status: q.status || { $ne: "deleted" } };
  if (q.roleId) filter.roles = q.roleId;           // mảng chứa phần tử roleId
  if (q.q) {
    const re = { $regex: escapeRegex(String(q.q).trim()), $options: "i" };
    filter.$or = [{ fullName: re }, { email: re }];
  }

  const [items, totalItems] = await Promise.all([
    User.find(filter).sort({ ...sort, _id: 1 }).skip((page - 1) *
limit).limit(limit).populate("roles"),
    User.countDocuments(filter),
  ]);
  res.json({ data: items.map(userView), meta: pageMeta(page, limit, totalItems)
});
});
```

Giải thích:

- `filter.roles = q.roleId`: với trường mảng, MongoDB khớp khi mảng **chứa** giá trị.
- `escapeRegex`: không có bước này, từ khóa như `(((` gây lỗi `500`, còn biểu thức phức tạp có thể làm treo truy vấn (ReDoS).

- Thêm `_id: 1` vào `sort` để thứ tự ổn định khi nhiều bản ghi có cùng giá trị sắp xếp — tránh trùng/sót bản ghi giữa các trang.

2.4. Implement POST /users

```
const PASSWORD_RE = /^(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$/;


router.post("/", requirePermission("users:create"), async (req, res) => {
  const body = req.body || {};
  // 1. Allow-list: chỉ nhận các trường này
  const details = notAllowedFields(body, ["email", "fullName", "phone",
"password", "roleIds"]);
  if (!body.password) details.push(detail("password", "REQUIRED", "Mật khẩu không
được để trống."));
  else if (!PASSWORD_RE.test(body.password)) details.push(detail("password",
"WEAK_PASSWORD", "Mật khẩu tối thiểu 8 ký tự, có chữ hoa, chữ số và ký tự đặc
biệt."));
  // TODO: roleIds (nếu có) phải là mảng ObjectId hợp lệ
  if (details.length) return fail(res, 400, "VALIDATION_ERROR", "Dữ liệu đầu vào
không hợp lệ.", details);


  // 2. Vai trò: có roleIds -> cần roles:assign, mọi role phải tồn tại, không vượt
quyền người gọi
  let roles = [];
  const roleIds = [...new Set(body.roleIds || [])];
  if (roleIds.length) {
    if (!req.permissions.has("roles:assign")) return fail(res, 403, "FORBIDDEN",
"Gán role khi tạo người dùng cần quyền roles:assign.");
    roles = await Role.find({ _id: { $in: roleIds } });
    if (roles.length !== roleIds.length) return fail(res, 400, "ROLE_NOT_FOUND",
"Có role không tồn tại trong roleIds.");
    // TODO: dùng missingPermissions(...) -> 403 PRIVILEGE_ESCALATION
  } else {
    roles = await Role.find({ code: "member" }); // mặc định
  }


  // 3. Email trùng -> 409 (kiểm tra trước để trả lỗi rõ ràng; unique index là
lưới an toàn)
  if (body.email && (await User.exists({ email:
String(body.email).trim().toLowerCase() })))
    return fail(res, 409, "EMAIL_ALREADY_EXISTS", "Email đã được sử dụng.");


  // 4. Tạo - lỗi validation của schema được error handler chuyển thành 400
  const user = await User.create({
    email: body.email,
    fullName: body.fullName,
    phone: body.phone ?? null,
    passwordHash: await bcrypt.hash(body.password, 10),
    roles: roles.map((r) => r._id),
  });
  await user.populate("roles");
  res.status(201).location(`${req.baseUrl}/${user._id}`).json(userView(user));
});
```

2.5. Implement PATCH /users/:userId

Yêu cầu (tự viết):

1. Allow-list `fullName`, `phone`, `email`, `status`; trường khác → `400 FIELD_NOT_ALLOWED`.
2. `status` chỉ nhận `active` | `locked`.

3. Khi chuyển sang `locked`: chính mình → `403 SELF_ACTION_FORBIDDEN`; super-admin cuối cùng → `409 LAST_SUPER_ADMIN`.
4. Đổi email sang email đã có → `409 EMAIL_ALREADY_EXISTS`.
5. Chỉ gán **những trường có trong body**, rồi gọi `await user.save()`.

```
// Gợi ý bước 5 — không ghi null cho trường thiếu
for (const k of ["fullName", "phone", "email", "status"]) if (k in body) user[k] =
body[k];
await user.save(); // save() chạy lại validator của schema
res.json(userView(user));
```

> Dùng `save()` thay cho `findByIdAndUpdate()` để validator của schema luôn chạy và dễ kiểm tra quy tắc nghiệp vụ trước khi ghi. Nếu dùng `findByIdAndUpdate()`, bắt buộc có `{ new: true, runValidators: true }`.

2.6. Implement DELETE /users/:userId (xóa mềm)

Tự viết: chặn tự xóa (`403`), chặn xóa super-admin cuối cùng (`409`), đặt `status = "deleted"`, `save()`, trả `res.status(204).end()`.

Câu hỏi: Vì sao xóa mềm thay vì `findByIdAndDelete()`? (Gợi ý: nhật ký, ràng buộc tham chiếu, khôi phục.)

2.7. Gán / gỡ vai trò

```
// PUT /users/:userId/roles/:roleId — idempotent
router.put("/:userId/roles/:roleId", requirePermission("roles:assign"), loadUser,
loadRole, async (req, res) => {
  const missing = missingPermissions(req.role.permissions, req.permissions);
  if (missing.length)
    return fail(res, 403, "PRIVILEGE_ESCALATION", `Không thể gán role
'${req.role.code}' vì bạn không có quyền: ${missing.join(", ")}.`);
  await User.updateOne({ _id: req.target._id }, { $addToSet: { roles: req.role._id
} }); // không tạo bản trùng
  res.status(204).end();
});
```

Tự viết `DELETE /users/:userId/roles/:roleId`: chặn tự gỡ (`403`), chống leo thang (`403`), gỡ `super-admin` khỏi super-admin cuối cùng (`409`), dùng `$pull`, trả `204`.

Giải thích: `$addToSet` chỉ thêm khi phần tử chưa có → gọi bao nhiêu lần kết quả vẫn như nhau (**idempotent**). Đây là lý do dùng `PUT` tới URI xác định thay vì `POST /addRoleToUser`.

2.8. routes/roles.js và routes/permissions.js

Tự viết theo đặc tả mục II.3.9–3.11. Gợi ý:

```
// Số người dùng (chưa bị xóa) đang mang role
const countUsers = (roleId) => User.countDocuments({ roles: roleId, status: { $ne:
"deleted" } });


// Kiểm tra mảng mã quyền: đúng kiểu và tồn tại trong danh mục
async function checkPermissionList(perms) {
  if (!Array.isArray(perms) || !perms.every((p) => typeof p === "string"))
```

```
    return { status: 400, code: "VALIDATION_ERROR", details:
[detail("permissions", "INVALID_TYPE", "permissions phải là mảng mã quyền.")] };
  const known = new Set((await Permission.find({ code: { $in: perms } })).map((p)
=> p.code));
  const unknown = perms.filter((p) => !known.has(p));
  if (unknown.length)
    return { status: 400, code: "PERMISSION_NOT_FOUND", details: unknown.map((p)
=> detail("permissions", "NOT_FOUND", `Quyền '${p}' không tồn tại.`)) };
  return null;
}
```

Danh sách việc cần làm:

- `GET /roles`: phân trang, `q` tìm trong `name`/`code`; tính `userCount` cho từng role bằng `Promise.all(roles.map((r) => countUsers(r._id)))`.
- `POST /roles`: allow-list; `checkPermissionList`; chống leo thang (mọi quyền cấp phải nằm trong `req.permissions`); trùng `code` → `409`; trả `201` + `Location`.
- `PATCH /roles/:roleId`: chỉ `name`, `description`.
- `DELETE /roles/:roleId`: hệ thống → `409 SYSTEM_ROLE_PROTECTED`; `countUsers > 0` → `409 ROLE_IN_USE`; `deleteOne()`; `204`.
- `PUT /roles/:roleId/permissions`: kiểm tra danh sách; hệ thống → `409`; chống leo thang trên **phần thay đổi** (quyền thêm vào và quyền bị bớt đi); lưu; `200`.
- `GET /permissions`: lọc `resource`, sắp xếp `resource`, `code`.

2.9. Bổ sung thêm

- Hoàn thiện TODO trong error handler: phân biệt `EMAIL_ALREADY_EXISTS` / `ROLE_CODE_EXISTS` theo `err.keyPattern`.
- Mọi endpoint trả đúng mã trạng thái và mã lỗi như đặc tả; không route nào trả `200` kèm `success: false`.
- Không có endpoint nào trả `passwordHash`; không có endpoint nào nhận `role`/`roles` qua `PATCH /users/:userId`.
- Không có `if (role === "...")` nào trong code: mọi kiểm tra qua `requirePermission` hoặc `req.permissions`.
- Gọi `GET` trên URI không tồn tại → `404` dạng JSON (không phải trang HTML mặc định).

Bước 3: Frontend (120 phút)

3.1. Tạo file index.html

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Quản lý Người dùng & Phân quyền</title>
```

```
  <!-- React CDN -->
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  <!-- Axios CDN -->
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
  <style>
    /* TODO: Thêm CSS (mục 3.11) */
  </style>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    // TODO: api, components
  </script>
</body>
</html>
```

> **Mở bằng VS Code Live Server** (`http://127.0.0.1:5500`), không mở trực tiếp file (`file://`): origin `null` sẽ bị CORS chặn.

3.2. Component Structure

```
App                          (me, toast, tab)
├── LoginPage                (khi chưa đăng nhập)
├── Header + Tabs            (hiển thị theo quyền)
├── UsersPage
│   ├── SearchBar            (debounce)
│   ├── bộ lọc trạng thái / vai trò
│   ├── UserTable            (nút theo quyền)
│   ├── Pagination
│   ├── UserModal            (thêm / sửa)
│   └── RoleAssignModal      (gán / gỡ vai trò, Promise.allSettled)
├── RolesPage
│   └── RoleModal            (thêm / sửa, checkbox quyền theo nhóm)
├── MyProfile
└── Toast
```

3.3. Axios instance và interceptor

```
const { useState, useEffect, useCallback, useRef } = React;

const API_BASE = "http://localhost:3001/api/v1";
// Làm Frontend trước với mock server của giảng viên:
// const API_BASE = "<MOCK_URL>/good/api/v1";

const TOKEN_KEY = "rbac_token";
const getToken = () => sessionStorage.getItem(TOKEN_KEY);
const setToken = (t) => (t ? sessionStorage.setItem(TOKEN_KEY, t) :
sessionStorage.removeItem(TOKEN_KEY));

const api = axios.create({ baseURL: API_BASE, timeout: 10000 });

// Request interceptor: tự gắn Bearer token cho MỌI request
api.interceptors.request.use((config) => {
  const token = getToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response interceptor: chuẩn hóa lỗi + xử lý 401 tập trung
api.interceptors.response.use(
  (res) => res,
```

```
  (error) => {
    if (!error.response) {
      return Promise.reject({
        status: 0, code: "NETWORK_ERROR", details: [],
        message: error.code === "ECONNABORTED" ? "Máy chủ phản hồi quá lâu." :
"Không kết nối được máy chủ (kiểm tra server, mạng hoặc CORS).",
      });
    }
    const { status, data } = error.response;
    const err = {
      status,
      code: data?.error?.code || `HTTP_${status}`,
      message: data?.error?.message || `Lỗi ${status}`,
      details: data?.error?.details || [],
    };
    if (status === 401 && !error.config.url.endsWith("/auth/tokens")) {
      setToken(null);
      window.dispatchEvent(new Event("auth:expired")); // App lắng nghe để về màn
hình đăng nhập
    }
    return Promise.reject(err);
  }
);

// Kiểm tra quyền phía giao diện (chỉ để ẩn/hiện)
const can = (me, perm) => !!me && me.permissions.includes(perm);
const sameId = (a, b) => String(a) === String(b);
```

Giải thích:

- Nhờ interceptor, các component chỉ cần `await api.get("/users")`; không phải tự gắn token hay tự phân loại lỗi.
- Sau interceptor, mọi lỗi có cùng dạng `{ status, code, message, details }` → `catch (err)` xử lý thống nhất.
- Token lưu trong `sessionStorage` (mất khi đóng tab). Lưu trong `localStorage` tiện hơn nhưng dễ bị đánh cắp nếu trang có lỗ hổng XSS — hãy thảo luận đánh đổi này khi vấn đáp.

3.4. Implement Component App

```
function App() {
  const [me, setMe] = useState(null);
  const [checking, setChecking] = useState(!!getToken());
  const [tab, setTab] = useState("users");
  const [toast, setToast] = useState(null);
  const toastTimer = useRef(null);

  const notify = useCallback((type, text) => {
    setToast({ type, text });
    clearTimeout(toastTimer.current); // hủy hẹn giờ của thông báo trước
    toastTimer.current = setTimeout(() => setToast(null), 4000);
  }, []);

  const loadMe = useCallback(async () => {
    const { data } = await api.get("/me");
    setMe(data);
    return data;
  }, []);
```

```
  // Còn token trong sessionStorage (F5 trang) -> thử lấy lại /me
  useEffect(() => {
    if (!getToken()) return;
    loadMe().catch(() => {}).finally(() => setChecking(false));
  }, [loadMe]);

  // Interceptor báo 401 -> về màn hình đăng nhập
  useEffect(() => {
    const onExpired = () => { setMe(null); notify("error", "Phiên đăng nhập đã hết
hạn. Vui lòng đăng nhập lại."); };
    window.addEventListener("auth:expired", onExpired);
    return () => window.removeEventListener("auth:expired", onExpired);
  }, [notify]);

  const logout = () => { setToken(null); setMe(null); };

  if (checking) return <div className="container">Đang tải…</div>;

  const tabs = [
    can(me, "users:read") && { key: "users", label: "Người dùng" },
    can(me, "roles:read") && { key: "roles", label: "Vai trò" },
    me && { key: "me", label: "Hồ sơ của tôi" },
  ].filter(Boolean);
  const current = tabs.some((t) => t.key === tab) ? tab : tabs[0]?.key;

  return (
    <div className="container">
      {!me ? (
        <LoginPage onLoggedIn={loadMe} />
      ) : (
        <>
          <div className="header">
            <div>
              <h1>Quản lý Người dùng &amp;amp; Phân quyền</h1>
              <div className="muted">
                {me.fullName} ({me.email}) · {me.roles.map((r) => <span key={r.id}
className="badge">{r.name}</span>)}
              </div>
            </div>
            <button onClick={logout}>Đăng xuất</button>
          </div>
          <div className="tabs">
            {tabs.map((t) => (
              <button key={t.key} className={current === t.key ? "active" : ""}
onClick={() => setTab(t.key)}>{t.label}</button>
            ))}
          </div>
          {current === "users" && <UsersPage me={me} notify={notify}
reloadMe={loadMe} />}
          {current === "roles" && <RolesPage me={me} notify={notify}
reloadMe={loadMe} />}
          {current === "me" && <MyProfile me={me} />}
        </>
      )}
      {toast && <div className={`toast ${toast.type}`}>{toast.text}</div>}
    </div>
  );
}
```

3.5. Implement Component LoginPage

```
function LoginPage({ onLoggedIn }) {
  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
```

```
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!form.email.trim() || !form.password) return setError("Vui lòng nhập email
và mật khẩu.");
    setLoading(true);
    setError("");
    try {
      const { data } = await api.post("/auth/tokens", { grantType: "password",
email: form.email.trim(), password: form.password });
      setToken(data.accessToken);
      await onLoggedIn();
    } catch (err) {
      setError(err.message); // 401 sai thông tin, 403 bị khóa, 0 lỗi mạng
    } finally {
      setLoading(false);
    }
  };

  return (
    <form className="login" onSubmit={handleSubmit}>
      <h1>Đăng nhập</h1>
      <label>Email</label>
      <input name="email" type="email" value={form.email} onChange={(e) =>
setForm({ ...form, email: e.target.value })} />
      <label>Mật khẩu</label>
      <input name="password" type="password" value={form.password} onChange={(e)
=> setForm({ ...form, password: e.target.value })} />
      {error && <p className="error">{error}</p>}
      <div className="modal-actions">
        <button className="primary" type="submit" disabled={loading}>{loading ?
"Đang đăng nhập…" : "Đăng nhập"}</button>
      </div>
    </form>
  );
}
```

3.6. Implement Component UsersPage

```
function UsersPage({ me, notify, reloadMe }) {
  const [users, setUsers] = useState([]);
  const [meta, setMeta] = useState({ page: 1, limit: 5, totalItems: 0, totalPages:
1 });
  const [page, setPage] = useState(1);
  const [limit, setLimit] = useState(5);
  const [search, setSearch] = useState("");
  const [status, setStatus] = useState("");
  const [roleId, setRoleId] = useState("");
  const [roles, setRoles] = useState([]);
  const [editing, setEditing] = useState(undefined); // undefined: đóng, null:
thêm mới, object: sửa
  const [assigning, setAssigning] = useState(null);

  const handleError = useCallback((err) => {
    notify("error", err.message);
    if (err.status === 403) reloadMe(); // quyền có thể vừa bị thay đổi -> đồng bộ
giao diện
  }, [notify, reloadMe]);

  const fetchUsers = useCallback(async () => {
    const params = { page, limit, sort: "-createdAt" };
    if (search.trim().length >= 2) params.q = search.trim(); // backend yêu cầu ≥
2 ký tự
    if (status) params.status = status;
```

```
    if (roleId) params.roleId = roleId;
    try {
      const { data } = await api.get("/users", { params });
      setUsers(data.data);
      setMeta(data.meta);
      if (page > data.meta.totalPages) setPage(data.meta.totalPages); // vd. xóa
dòng cuối của trang cuối
    } catch (err) {
      handleError(err);
    }
  }, [page, limit, search, status, roleId, handleError]);

  useEffect(() => { fetchUsers(); }, [fetchUsers]);

  // Danh sách role cho bộ lọc và modal
  useEffect(() => {
    if (!can(me, "roles:read")) return;
    (async () => {
      try {
        const { data } = await api.get("/roles", { params: { limit: 100 } });
        setRoles(data.data);
      } catch (err) { handleError(err); }
    })();
  }, [me, handleError]);

  // Đổi bộ lọc / số dòng -> quay về trang 1
  const changeFilter = (setter) => (value) => { setter(value); setPage(1); };

  const toggleLock = async (user) => {
    // TODO: PATCH /users/:id { status: "locked" | "active" }, thông báo,
fetchUsers()
  };

  const deleteUser = async (user) => {
    if (!window.confirm(`Bạn có chắc muốn xóa "${user.fullName}"?`)) return;
    // TODO: DELETE /users/:id (204, không có body), thông báo, fetchUsers()
  };

  return (
    <div>
      <div className="toolbar">
        <SearchBar onSearch={changeFilter(setSearch)} />
        {/* TODO: select trạng thái, select vai trò (khi có roles:read), nút Thêm
(khi có users:create) */}
      </div>
      <UserTable users={users} me={me} offset={(meta.page - 1) * meta.limit}
        onEdit={setEditing} onAssign={setAssigning} onToggleLock={toggleLock}
onDelete={deleteUser} />
      <Pagination meta={meta} limit={limit} onPageChange={setPage}
onLimitChange={changeFilter(setLimit)} />
      {editing !== undefined && (
        <UserModal user={editing} roles={roles} me={me} notify={notify}
          onClose={() => setEditing(undefined)} onSaved={() => {
setEditing(undefined); fetchUsers(); }} />
      )}
      {assigning && (
        <RoleAssignModal user={assigning} roles={roles} me={me} notify={notify}
          onClose={() => setAssigning(null)} onSaved={() => { setAssigning(null);
fetchUsers(); }} />
      )}
    </div>
  );
}
```

SearchBar (debounce 400ms):

```
function SearchBar({ onSearch }) {
  const [input, setInput] = useState("");
  useEffect(() => {
    const timer = setTimeout(() => onSearch(input), 400);
    return () => clearTimeout(timer);
  }, [input]);
  return <input placeholder="Tìm theo họ tên, email (tối thiểu 2 ký tự)…"
value={input} onChange={(e) => setInput(e.target.value)} />;
}
```

3.7. Implement Component UserTable

```
function UserTable({ users, me, offset, onEdit, onAssign, onToggleLock, onDelete
}) {
  return (
    <table>
      <thead>
        <tr><th>STT</th><th>Họ tên</th><th>Email</th><th>Vai trò</th><th>Trạng
thái</th><th>Thao tác</th></tr>
      </thead>
      <tbody>
        {users.length === 0 ? (
          <tr><td colSpan="6" style={{ textAlign: "center" }}>Không có dữ
liệu</td></tr>
        ) : users.map((u, i) => {
          const isSelf = sameId(u.id, me.id);
          return (
            <tr key={u.id}>
              <td>{offset + i + 1}</td>
              <td>{u.fullName}{isSelf && " (bạn)"}</td>
              <td>{u.email}</td>
              <td>{u.roles.map((r) => <span key={r.id}
className="badge">{r.name}</span>)}</td>
              <td><span className={`badge ${u.status}`}>{u.status === "locked" ?
"Đã khóa" : "Hoạt động"}</span></td>
              <td>
                {can(me, "users:update") && <button onClick={() =>
onEdit(u)}>Sửa</button>}
                {can(me, "roles:assign") && can(me, "roles:read") && <button
onClick={() => onAssign(u)}>Vai trò</button>}
                {can(me, "users:update") && (
                  <button disabled={isSelf} onClick={() =>
onToggleLock(u)}>{u.status === "locked" ? "Mở khóa" : "Khóa"}</button>
                )}
                {can(me, "users:delete") && <button className="danger"
disabled={isSelf} onClick={() => onDelete(u)}>Xóa</button>}
              </td>
            </tr>
          );
        })}
      </tbody>
    </table>
  );
}
```

Pagination: tái sử dụng component của Ex3, sửa để nhận `meta` (`meta.page`, `meta.totalPages`, `meta.totalItems`), dropdown 5/10/20.

3.8. Implement Component UserModal

```
function UserModal({ user, roles, me, notify, onClose, onSaved }) {
  const isNew = !user;
```

```
  const [form, setForm] = useState({
    email: user?.email || "", fullName: user?.fullName || "", phone: user?.phone
|| "", password: "", roleIds: [],
  });
  const [errors, setErrors] = useState({});

  const validate = () => {
    const e = {};
    if (!/^\S+@\S+\.\S+$/.test(form.email.trim())) e.email = "Email không hợp lệ";
    if (form.fullName.trim().length < 2) e.fullName = "Họ tên phải có ít nhất 2 ký
tự";
    // TODO: phone (nếu có) dạng +84..., password (khi thêm mới) đủ mạnh
    setErrors(e);
    return Object.keys(e).length === 0;
  };

  const handleSubmit = async () => {
    if (!validate()) return;
    const clean = { email: form.email.trim().toLowerCase(), fullName:
form.fullName.trim(), phone: form.phone.trim() || null };
    try {
      if (isNew) {
        // TODO: POST /users với clean + password (+ roleIds nếu có chọn)
      } else {
        // PATCH: chỉ gửi trường thay đổi
        const changed = Object.fromEntries(Object.entries(clean).filter(([k, v])
=> v !== (user[k] ?? null)));
        if (Object.keys(changed).length) await api.patch(`/users/${user.id}`,
changed);
      }
      notify("success", isNew ? "Tạo người dùng thành công." : "Cập nhật người
dùng thành công.");
      onSaved();
    } catch (err) {
      // 400: lỗi theo từng trường; 409: email trùng
      const fieldErrors = {};
      err.details.forEach((d) => { if (d.field) fieldErrors[d.field] = d.message;
});
      if (err.code === "EMAIL_ALREADY_EXISTS") fieldErrors.email = err.message;
      setErrors(fieldErrors);
      if (!Object.keys(fieldErrors).length) notify("error", err.message);
    }
  };

  // TODO: render modal theo wireframe 4.4
  //  - ô Mật khẩu chỉ khi isNew
  //  - checkbox vai trò chỉ khi isNew && can(me, "roles:assign");
  //    disabled khi role.permissions có quyền mà me.permissions không có
}
```

3.9. Implement Component RoleAssignModal

```
function RoleAssignModal({ user, roles, me, notify, onClose, onSaved }) {
  const initial = user.roles.map((r) => String(r.id));
  const [selected, setSelected] = useState(initial);
  const [saving, setSaving] = useState(false);
  const isSelf = sameId(user.id, me.id);

  const toggle = (id) => setSelected(selected.includes(id) ? selected.filter((x)
=> x !== id) : [...selected, id]);

  const handleSave = async () => {
    const toAdd = selected.filter((id) => !initial.includes(id));
    const toRemove = initial.filter((id) => !selected.includes(id));
```

```
    const ops = [
      ...toAdd.map((id) => ({ id, action: "gán", run: () =>
api.put(`/users/${user.id}/roles/${id}`) })),
      ...toRemove.map((id) => ({ id, action: "gỡ", run: () =>
api.delete(`/users/${user.id}/roles/${id}`) })),
    ];
    if (!ops.length) return onClose();
    setSaving(true);
    // Chạy song song, chờ TẤT CẢ kết thúc dù có thao tác thất bại
    const results = await Promise.allSettled(ops.map((op) => op.run()));
    setSaving(false);
    const failed = results
      .map((r, i) => ({ r, op: ops[i] }))
      .filter(({ r }) => r.status === "rejected")
      .map(({ r, op }) => `Không thể ${op.action} "${roles.find((x) =>
sameId(x.id, op.id))?.name}": ${r.reason.message}`);
    if (failed.length) notify("error", failed.join(" "));
    else notify("success", `Đã cập nhật vai trò cho ${user.fullName}.`);
    onSaved();
  };

  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content" onClick={(e) => e.stopPropagation()}>
        <h3>Vai trò của {user.fullName}</h3>
        {roles.map((r) => {
          const id = String(r.id);
          const missing = r.permissions.filter((p) =>
!me.permissions.includes(p));
          const lockedSelf = isSelf && initial.includes(id); // không tự gỡ role
của mình
          return (
            <label key={id} className="check">
              <input type="checkbox" checked={selected.includes(id)}
disabled={missing.length > 0 || lockedSelf} onChange={() => toggle(id)} />
              <span>
                <b>{r.name}</b> <span className="muted">({r.code},
{r.permissions.length} quyền)</span>
                {missing.length > 0 && <span className="muted"> — vượt quyền của
bạn</span>}
                {lockedSelf && <span className="muted"> — không thể tự gỡ</span>}
              </span>
            </label>
          );
        })}
        <div className="modal-actions">
          <button className="primary" onClick={handleSave}
disabled={saving}>{saving ? "Đang lưu…" : "Lưu"}</button>
          <button onClick={onClose}>Hủy</button>
        </div>
      </div>
    </div>
  );
}
```

Giải thích: Với `Promise.all`, chỉ cần một request lỗi là toàn bộ bị coi là thất bại và mất kết quả các request khác. `Promise.allSettled` chờ tất cả kết thúc và cho biết từng request thành công hay thất bại — phù hợp khi các thao tác độc lập với nhau.

3.10. Implement RolesPage, RoleModal, MyProfile

Tự viết theo mục II.4.6 và II.4.7. Gợi ý:

```
// Nhóm danh mục quyền theo resource: { users: [...], roles: [...], permissions:
[...] }
const groups = catalog.reduce((acc, p) => ({ ...acc, [p.resource]:
[...(acc[p.resource] || []), p] }), {});

// Lưu khi sửa: PATCH nếu tên/mô tả đổi, rồi PUT permissions nếu bộ quyền đổi
const changed = perms.length !== role.permissions.length || perms.some((p) =>
!role.permissions.includes(p));
if (!role.isSystem && changed) await api.put(`/roles/${role.id}/permissions`, {
permissions: perms });
```

- `RolesPage` tải song song danh sách vai trò và danh mục quyền (khi có `permissions:read`).
- Sau khi lưu vai trò, gọi lại `reloadMe()`: nếu người đang đăng nhập mang vai trò vừa sửa, quyền của chính họ thay đổi.

3.11. App Render và CSS

```
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

CSS: tái sử dụng CSS của Ex3, bổ sung:

```
.header { display: flex; justify-content: space-between; align-items: center; gap:
12px; flex-wrap: wrap; margin-bottom: 16px; }
.muted { color: #666; font-size: 14px; }
.tabs { display: flex; gap: 4px; border-bottom: 2px solid #eee; margin-bottom:
16px; }
.tabs button.active { background: #4CAF50; color: #fff; }
.toolbar { display: flex; gap: 8px; flex-wrap: wrap; margin-bottom: 12px; }
.badge { display: inline-block; padding: 2px 8px; margin: 2px; border-radius:
10px; font-size: 12px; background: #e3f2fd; color: #1565c0; }
.badge.locked { background: #ffebee; color: #c62828; }
.badge.active { background: #e8f5e9; color: #2e7d32; }
.badge.system { background: #fff3e0; color: #e65100; }
button.primary { background: #4CAF50; color: #fff; }
button.danger { background: #e53935; color: #fff; }
button:disabled { opacity: .5; cursor: not-allowed; }
.check { display: flex; align-items: flex-start; gap: 8px; font-weight: normal;
margin-top: 6px; }
.toast { position: fixed; right: 16px; bottom: 16px; padding: 12px 16px; border-
radius: 6px; color: #fff; max-width: 360px; }
.toast.success { background: #2e7d32; }
.toast.error { background: #c62828; }
```

3.12. Yêu cầu bổ sung

- Không dùng `.then()`; mọi lời gọi API dùng `async/await` + `try/catch`.
- Không tự gắn header `Authorization` trong component — chỉ ở request interceptor.
- Không cập nhật state danh sách trực tiếp sau CRUD — luôn gọi lại API.
- Khi xóa bản ghi cuối cùng của trang cuối, tự lùi về trang trước.
- Nút "Lưu" bị disabled trong lúc đang gửi request (tránh bấm hai lần).
- Kiểm tra Frontend với **cả** Backend của mình và mock server của giảng viên: chỉ đổi `API_BASE`, không sửa chỗ nào khác.

IV. KIỂM THỬ, SẢN PHẨM NỘP VÀ ĐÁNH GIÁ

1. Test cases bắt buộc (Postman)

Chuẩn bị 4 token: `super-admin`, `user-manager`, `auditor`, `member`.

#	Request	Token	Kỳ vọng
T01	`GET /users`	không có	`401 UNAUTHENTICATED`
T02	`GET /users`	member	`403 FORBIDDEN`
T03	`GET /users?page=2&limit=3&sort=fullName`	auditor	`200`, `meta` đúng
T04	`GET /users?q=nguyen&status=active`	auditor	`200`, chỉ bản ghi khớp
T05	`GET /users?limit=500`	auditor	`400 INVALID_QUERY_PARAMETER`
T06	`POST /users` hợp lệ	user-manager	`201`, có `Location`, không có `passwordHash`
T07	`POST /users` trùng email	user-manager	`409 EMAIL_ALREADY_EXISTS`
T08	`POST /users` mật khẩu `123`	user-manager	`400`, `issue: WEAK_PASSWORD`
T09	`POST /users` kèm `roleIds: [<super-admin>]`	user-manager	`403 PRIVILEGE_ESCALATION`
T10	`PATCH /users/:id { "phone": "+84911222333" }`	user-manager	`200`, các trường khác giữ nguyên
T11	`PATCH /users/:id { "roles": [...] }`	user-manager	`400 FIELD_NOT_ALLOWED`
T12	`PATCH /users/<chính mình> { "status": "locked" }`	user-manager	`403 SELF_ACTION_FORBIDDEN`
T13	`PUT /users/:id/roles/<auditor>` gọi 2 lần	user-manager	`204`, `204`; vai trò không bị trùng
T14	`PUT /users/:id/roles/<super-admin>`	user-manager	`403 PRIVILEGE_ESCALATION`
T15	Gỡ `user-manager` của A (bằng token super-admin), rồi A gọi `GET /users` bằng **token cũ**	A	`403` ngay lập tức
T16	`DELETE /users/:id`	user-manager	`403 FORBIDDEN`
T17	`DELETE /users/:id`	super-admin	`204`; sau đó `GET` → `404`, đăng nhập → `401`
T18	`GET /users/abc`	super-admin	`404 USER_NOT_FOUND` (không phải `500`)
T19	`POST /roles` mã trùng	super-admin	`409 ROLE_CODE_EXISTS`

#	Request	Token	Kỳ vọng
T20	`PUT /roles/<member>/permissions`	super-admin	`409 SYSTEM_ROLE_PROTECTED`
T21	`DELETE /roles/<role đang dùng>`	super-admin	`409 ROLE_IN_USE`
T22	`PUT /roles/:id/permissions` có `"users:fly"`	super-admin	`400 PERMISSION_NOT_FOUND`
T23	Gửi kèm header tự đặt `X-User-Role: admin` với token member	member	vẫn `403` (BR-09)
T24	`POST /users` body là JSON hỏng	user-manager	`400 MALFORMED_JSON`

2. Kiểm thử tích hợp và bảo mật Frontend

- Đăng nhập lần lượt 4 tài khoản: các tab, nút hiển thị đúng bảng ở mục II.4.2.
- Mở DevTools → tab **Network**: mọi request có `Authorization: Bearer …`; request `PATCH`/`PUT`/`DELETE` có preflight `OPTIONS` thành công.
- Xóa token trong *Application → Session Storage* rồi thao tác: giao diện quay về màn hình đăng nhập.
- Dùng tài khoản member, gọi trực tiếp `fetch` trong Console tới `POST /users`: Backend trả `403` dù giao diện không có nút.

3. Sản phẩm nộp

- Mã nguồn `backend/` (không kèm `node_modules`, `.env`) và `frontend/index.html`.
- File `.env.example` (các biến, không có giá trị thật).
- Postman collection (`.json`) chứa T01–T24, mỗi request có test kiểm tra mã trạng thái.
- Ảnh chụp giao diện khi đăng nhập bằng 3 vai trò khác nhau.

V. PHỤ LỤC

1. Danh sách mã lỗi

HTTP	`error.code`	Ý nghĩa
400	`VALIDATION_ERROR`	Dữ liệu body không hợp lệ (xem `details`)
400	`MALFORMED_JSON`	Body không phải JSON hợp lệ
400	`INVALID_QUERY_PARAMETER`	Tham số query sai
400	`ROLE_NOT_FOUND`	`roleIds` trong body chứa role không tồn tại
400	`PERMISSION_NOT_FOUND`	Danh sách quyền chứa mã không tồn tại
401	`UNAUTHENTICATED`	Thiếu token
401	`INVALID_TOKEN`	Token sai, hoặc người dùng không còn hoạt động
401	`TOKEN_EXPIRED`	Token hết hạn
401	`INVALID_CREDENTIALS`	Sai email hoặc mật khẩu
403	`FORBIDDEN`	Thiếu permission
403	`PRIVILEGE_ESCALATION`	Gán/cấp quyền vượt quá quyền của mình
403	`SELF_ACTION_FORBIDDEN`	Tự khóa/xóa/gỡ vai trò của chính mình
403	`ACCOUNT_LOCKED`	Tài khoản bị khóa
404	`USER_NOT_FOUND`, `ROLE_NOT_FOUND`	Tài nguyên trên URI không tồn tại
404	`ENDPOINT_NOT_FOUND`	Endpoint không tồn tại
409	`EMAIL_ALREADY_EXISTS`, `ROLE_CODE_EXISTS`	Trùng dữ liệu
409	`ROLE_IN_USE`	Vai trò đang được gán
409	`SYSTEM_ROLE_PROTECTED`	Vai trò hệ thống
409	`LAST_SUPER_ADMIN`	Super-admin cuối cùng
500	`INTERNAL_ERROR`	Lỗi phía server

2. Lỗi thường gặp khi làm bài

Hiện tượng	Nguyên nhân thường gặp
Frontend báo "Không kết nối được máy chủ" dù server đang chạy	Origin của trang chưa có trong `CORS_ORIGINS`; mở file bằng `file://`
Mọi request sau đăng nhập đều `401`	Quên request interceptor; header thiếu chữ `Bearer` ; `JWT_SECRET` khác nhau giữa lúc ký và lúc kiểm tra
`GET /users/abc` trả `500`	Không kiểm tra ObjectId trước khi truy vấn (`CastError`)
Trùng email trả `500`	Error handler chưa xử lý `err.code === 11000`

Hiện tượng	Nguyên nhân thường gặp
Tìm kiếm với `(` trả `500`	Chưa thoát ký tự đặc biệt cho `$regex`
Gỡ vai trò nhưng người dùng vẫn thao tác được	Lấy quyền từ token hoặc cache thay vì tra lại CSDL
Sửa họ tên làm mất số điện thoại	Ghi đè toàn bộ document thay vì chỉ trường được gửi
`roles` trong response là mảng ObjectId	Quên `.populate("roles")`